

TriangleFX

From Three Linear Constraints to a Verified Triangle

CSC360 – Group 1

Software Design and Implementation in Java / JavaFX

[Java 17](#) ▪ [JavaFX](#) ▪ [Maven](#) ▪ [JUnit 5](#)

1. Problem Statement

The problem

Given **three lines** in the plane, decide whether they bound a real triangle. If they do, compute and **draw** it.

Requirements

- Accept input as the matrix system $A\mathbf{x} = \mathbf{b}$
- Find the three vertices (pairwise intersections)
- Reject impossible geometry with a *clear reason*
- Show area, perimeter, sides and centroid
- Render on a scaled coordinate plane

Inputs that must fail gracefully

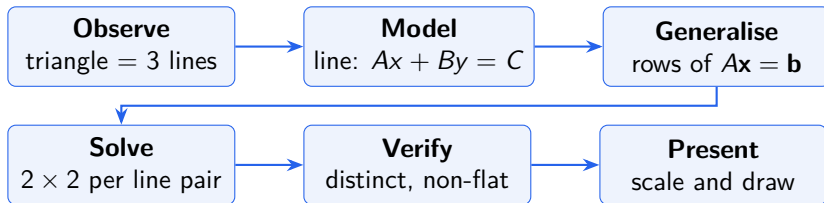
- Two lines parallel
- Two lines identical
- All three lines through one point
- Vertices collinear (zero area)
- Malformed / missing numbers

Example

$$x + y = 8, x - y = 2, x = 1$$

$\Rightarrow$ vertices $(5, 3), (1, -1), (1, 7)$, area 16

2. Thought Process



Key decisions

- Matrix input is unambiguous; no free-text algebra needed in the UI
- Keep math **independent of JavaFX** so it is unit-testable
- Treat invalid geometry as a first-class, explained outcome

Alternatives considered

- Parsing typed equations ($x + y = 8$): kept as an optional `EquationParser`, not the primary path
- Solving a 3×2 system directly: over-determined in general, so we solve *pairs* instead

3. The Mathematics

Matrix form. Each row of A is one line:

$$\begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \\ a_{31} & a_{32} \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}$$

Intersection (Cramer's rule) for
 $L_1 : A_1x + B_1y = C_1$, $L_2 : A_2x + B_2y = C_2$:

$$D = A_1B_2 - A_2B_1$$

$$x = \frac{C_1B_2 - C_2B_1}{D}, \quad y = \frac{A_1C_2 - A_2C_1}{D}$$

$D = 0 \Rightarrow$ parallel or identical.

Vertices.

$$P_{12} = L_1 \cap L_2, \quad P_{23} = L_2 \cap L_3, \quad P_{31} = L_3 \cap L_1$$

Area (shoelace formula)

$$\Delta = \frac{1}{2} |x_1(y_2 - y_3) + x_2(y_3 - y_1) + x_3(y_1 - y_2)|$$

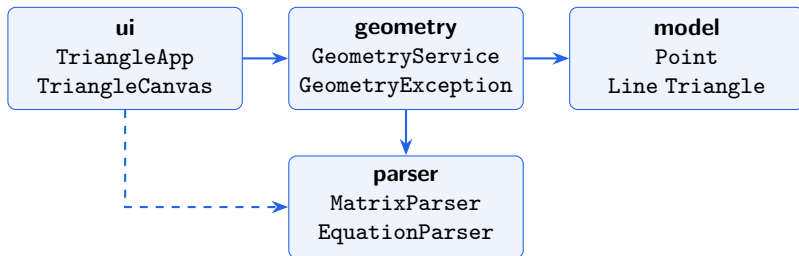
Centroid and perimeter

$$G = \left(\frac{x_1 + x_2 + x_3}{3}, \frac{y_1 + y_2 + y_3}{3} \right)$$

$$p = \|P_{12}P_{23}\| + \|P_{23}P_{31}\| + \|P_{31}P_{12}\|$$

Valid triangle $\iff \Delta \geq \varepsilon$ ($\varepsilon = 10^{-6}$).

4. The Solution: Layered Architecture



- **Dependencies point inward:** model knows nothing about UI or parsing
- Main only launches TriangleApp
- Pure logic lives in geometry/model: testable without a display
- ui only collects input and draws results

5. Solution Walk-through: Validation Pipeline

- 1 **Pair check** (3 pairs): identical or parallel $\rightarrow$ error
- 2 **Intersect** each pair (Cramer)
- 3 **Concurrency**: two vertices within tolerance $\rightarrow$ error
- 4 **Area**: if $\Delta < 10^{-6} \rightarrow$ error
- 5 Return the verified Triangle

Every failure carries a human-readable message naming the offending lines.

```
Triangle buildTriangle(Line l1, Line l2, Line l3)
    throws GeometryException {
    // 1. reject identical / parallel pairs
    checkPair(l1, l2, 1, 2);
    checkPair(l2, l3, 2, 3);
    checkPair(l3, l1, 3, 1);

    // 2. the three vertices
    Point p12 = l1.intersectionWith(l2).orElseThrow(..);
    Point p23 = l2.intersectionWith(l3).orElseThrow(..);
    Point p31 = l3.intersectionWith(l1).orElseThrow(..);

    // 3. concurrency: coincident vertices
    if (p12.equalsWithTolerance(p23, TOL) || ..)
        throw new GeometryException("concurrent");

    // 4. degenerate (near-zero area)
    Triangle t = new Triangle(p12, p23, p31, l1, l2, l3);
    if (t.getArea() < MIN_AREA)
        throw new GeometryException("collapsed");
    return t;
}
```

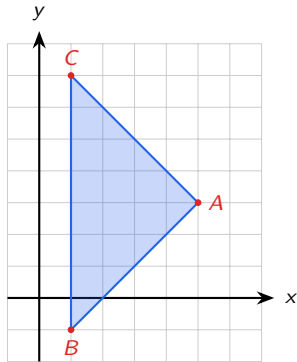
6. Rendering: World-to-Screen Mapping

The canvas fits the triangle to any window size using a single uniform scale, so shapes are never distorted:

$$X_{\text{px}} = \frac{w}{2} + (x - x_c) s$$

$$Y_{\text{px}} = \frac{h}{2} - (y - y_c) s$$

- (x_c, y_c) : centre of the bounding box; s : pixels per unit
- The minus sign flips y (screen y grows downward)
- Grid, axes, lines, fill, and labelled vertices drawn in order



Default example: $A(5, 3)$, $B(1, -1)$, $C(1, 7)$

7. Design Patterns and Principles in the Codebase

Pattern / principle	Where	Why it helps
Facade / service layer	GeometryService	One entry point hides parsing, pairing, intersection and validation
Immutable value objects	Point, Line, Triangle	final fields: thread-safe, safe to share with the UI
Optional for “no result”	Line.intersectionWith	Callers must handle parallel lines; no null
Fail-fast + exception translation	Line, GeometryService	Line rejects $A = B = 0$; service rewraps as a domain error
Static utility classes	MatrixParser, EquationParser	Private constructors; stateless, easy to test
Observer (JavaFX listeners)	TriangleCanvas	Size listeners call redraw() on resize
Separation of concerns	model, geometry, ui	Logic testable without a display

8. Engineering Quality

Numerical robustness

- Tolerances (10^{-9} for parallelism, 10^{-6} for area/points) instead of exact ==
- Normalises -0.0 in intersection results
- `Math.hypot` for the “ A, B not both zero” check

Two error families

`ParseException` (bad text) vs.
`GeometryException` (bad geometry): the UI reports each differently.

Tests (JUnit 5)

- `GeometryServiceTest`
- `EquationParserTest`
- `MatrixParserTest`

Cover valid triangles,
parallel/coincident/concurrent/degenerate
inputs and parser edge cases.

Documentation and build

Javadoc on the public API, Maven build,
dark-theme UI; design docs in docs/.

9. Reference Material, Lessons and Next Steps

Reference books (Books/)

- *Core Java, Vol. 1 & 2*: classes, immutability, exceptions, regular expressions
- *Introducing JavaFX 8 Programming* (Weaver): scene graph, Canvas, property listeners
- *Graphics with Java* (2007): 2D drawing and coordinate transformations
- *Digital Image Processing* (Burger & Burge): geometric coordinate mapping

Lessons learned

- Fix the input model first
- Floating point needs explicit tolerances
- Domain exceptions give clear UI messages

Next steps

- Property-based tests
- Export drawing as PNG/SVG
- Generalise to n lines

Thank you — Questions?